



Summarizer

30/01/2023
Ana Jesus



git add .



git commit



git push

Git



Git



Git pull origin master --allow-unrelated-histories:

When there is a conflict between files that two people are placing in the same repository and the **FATAL** error appears.

First Pull and then Push

Build

This file compiles and archives your plug-in source code into a single JAR file. The build.properties file controls what goes into your plug-in distribution.



Build Money Bank

```
<project name="MoneyBank" default="jarfile">
  <!-- Initialize build properties -->
  <target name="init" description="Initializes properties">
    <property name="project.name" value="MoneyBank" />
    <property name="src.dir" value="src" />
    <property name="main.class" value="org.academiadecodigo.bootcamp.MoneyBank.Main" />
    <property name="build.dir" value="build" />
    <property name="classes.dir" value="${build.dir}/classes" />
  </target>

  <!-- Creates the build directories to hold JAR and Class files -->
  <target name="prepare" description="Creates the build and classes directories" depends="init">
    <mkdir dir="${classes.dir}" />
  </target>

  <!-- Removes the build directory -->
  <target name="clean" description="Clean up project" depends="init">
    <delete dir="${build.dir}" />
  </target>

  <!-- Compiles the source code -->
  <target name="compile" description="Compiles the source code" depends="prepare">
    <javac srcdir="${src.dir}" destdir="${classes.dir}" />
  </target>

  <!-- Creates a JAR file -->
  <target name="jarfile" description="Archives the code" depends="compile">
    <jar destfile="${build.dir}/${project.name}.jar" basedir="${classes.dir}">
      <manifest>
        <attribute name="Main-Class" value="${main.class}" />
      </manifest>
    </jar>
  </target>
</project>
```

MoneyBank

Project

Difficulties:

Understand how to put into code what I thought.

How to make classes communicate with each other

How to figure out what syntax to use:

- if public or private
- if void or not



Money Bank - Wallet

```
public class Wallet {  
  
    private int walletMoney;  
  
    public Wallet (int putWallet, int takeWallet){  
        walletMoney = putWallet - takeWallet;  
        System.out.println("I have " + walletMoney + " in my Wallet");  
    }  
    public void addMoney(int addMoney){  
        walletMoney = walletMoney + addMoney;  
    }  
    public int getWalletMoney(){  
        return walletMoney;  
    }  
    public void takeWalletMoney(int money){  
        walletMoney = walletMoney - money;  
    }  
}
```

Money Bank - Piggy

```
public class PiggyBank {  
    private int piggyMoney;  
  
    public PiggyBank (int putBank, int takeBank){  
        piggyMoney = putBank - takeBank;  
        System.out.println("I have " + piggyMoney + " in my Piggy");  
    }  
  
    public void addMoney(int moneySave){  
        piggyMoney = piggyMoney + moneySave;  
    }  
  
    public int piggyMoney(){  
        return piggyMoney;  
    }  
}
```


Money Bank - Person

```
public class Person {  
    private String name;  
  
    public Wallet wallet = new Wallet(0,0);  
    public PiggyBank piggy = new PiggyBank( 0,0);  
  
    public Person (String name){  
        this.name = name;  
    }  
}
```

Money Bank - Person

```
public void spendMoney(int money){
    wallet.takeWalletMoney(money);
    //wallet.takeWalletMoney() = wallet.getWalletMoney() - money;
    System.out.println(name + " spend " + money + ". And have " + wallet.getWalletMoney() + " in the
    Wallet");
    System.out.println("Wallet now has " + wallet.getWalletMoney() + "€ in the wallet");
}

public void saveMoney(int moneySave){
    //piggy.piggyMoney = piggy.piggyMoney + moneySave;
    piggy.addMoney(moneySave);
    System.out.println(name + " save " + moneySave + " in the piggy");
    System.out.println("Piggy now has " + piggy.piggyMoney());
}

public void fillWallet(int moneyFill){
    wallet.addMoney(moneyFill);
    //wallet.walletMoney = moneyFill + wallet.walletMoney;
    System.out.println(name + " put " + moneyFill + " in the wallet");
    System.out.println("Wallet now has " + wallet.getWalletMoney());
}
}
```

Money Bank - Main

```
public class Main {  
    public static void main(String[] args) {  
  
        Person person = new Person("Ana");  
  
        person.saveMoney(60);  
        person.fillWallet(10);  
        person.spendMoney(20);  
  
    }  
}
```

Money Bank

```
/Library/Java/JavaVirtualMachines/openjdk-8.jdk/Contents/Home/bin/jav
```

```
I have 0 in my Wallet
```

```
I have 0 in my Piggy
```

```
Ana save 60 in the piggy
```

```
Piggy now has 60
```

```
Ana put 10 in the wallet
```

```
Wallet now has 10
```

```
Ana spend 20. And have -10 in the Wallet
```

```
Wallet now has -10€ in the wallet
```

```
Process finished with exit code 0
```

Thank you!!

